

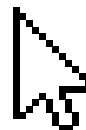


COE 454

Software Engineering II:

The Business of Software Engineering

IP · Contracts · Pricing · Client Relations · Career



Presented by **Eliel Keelson**



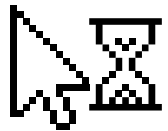
Intellectual Property & Copyright





Ghana Copyright Act, 2005 – Act 690

- **Software** is a literary work.
- Copyright belongs to the author from the moment of creation.
- No registration is required for copyright to exist.

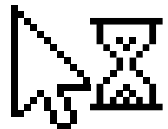




Ghana Copyright Act, 2005 – Act 690

What this means for you as a software engineer:

- Every line of code you write belongs to you by default, unless a contract says otherwise
- When a client pays you to build something, the copyright does NOT automatically transfer to them
- For the client to own the code, the contract must contain an explicit IP assignment clause
- If there is no signed contract, and no IP assignment clause, you retain ownership, even after delivery and payment
- Your team project this semester: seven people built a system together, you are all joint authors under Ghanaian law

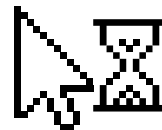




Moral Rights vs Economic Rights

Economic Rights – can be transferred:

- The right to reproduce the work (make copies)
- The right to distribute or sell the work
- The right to create derivative works based on it
- The right to license it to others for a fee
- These rights CAN be sold or assigned to a client via a written IP assignment clause

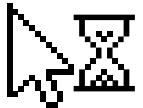




Moral Rights vs Economic Rights

Moral Rights – cannot be taken away:

- The right to be identified as the author of the work
- The right to object to derogatory treatment that harms your reputation
- These rights belong to you permanently under Ghanaian law
- A client cannot require you to waive these in a contract, even if they try
- Practical implication: your name stays on your work, even when the client owns the distribution rights

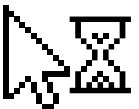




Open-Source Licensing: What You Are Building On

Safe to use in a commercial product:

- MIT License: use it, modify it, sell it. Keep the original copyright notice. Most npm packages use MIT.
- Apache 2.0: same as MIT, plus an explicit patent grant. Safe for commercial products.
- BSD (2-clause, 3-clause): permissive; minimal restrictions. Safe.
- Rule of thumb: if the license says 'do whatever you want', you are safe.

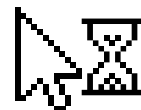




Open-Source Licensing: What You Are Building On

Requires careful handling:

- GNU GPL: if your distributed product contains GPL code, your entire product must also be open-sourced under the GPL. This ends your ability to sell a closed product.
- LGPL: slightly more permissive than GPL, linking is allowed, but modifying the library itself triggers open-source obligations.
- Practical step: before delivering to a client, run: `npx license-checker --summary` to audit every dependency.





Check out the slides on Open-Source
Licensing to get the full picture
of this concept.



—Very Important



IP in Practice: Two Real Scenarios

X **Common Mistake:**

- A developer builds a stock management system for a Kumasi shop. No contract is signed. The client pays in cash. A year later, the developer sells a similar system to a competitor. The client is furious and threatens legal action.
- Result: the developer was within their rights. No contract = no IP transfer. But the relationship is destroyed and the reputation damaged.
- The real mistake was not protecting either party with a written agreement from the start.

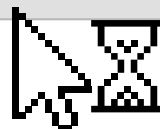




IP in Practice: Two Real Scenarios

✓ **Best Practice:**

- A developer presents a simple contract before starting any work. The contract specifies: client owns the code after final payment is made and confirmed. Developer retains the right to use non-client-specific components in future projects.
- Both parties sign. The scope, payment, and IP are clear from day one.
- When the client later wants to resell the product, they come back to negotiate, the developer has a second commercial conversation instead of a legal dispute.





Worth Thinking About (Intellectual Property)

- If your team's client project were commercialized after graduation, which of your seven team members would own the code, and in what proportion?
- Have you checked the licenses of every library your project depends on? Could any of them prevent commercial use?
- If a client asked you to sign an IP assignment clause giving them full ownership of the code, what would you need in return to make that acceptable?
- What would you do if you discovered, six months after delivery, that the client had sold your system to a third party without your knowledge?
- Does your current team charter say anything about who owns what you have built together?





Pricing Your Work





The Four Pricing Models

- **One-time license:** client pays once for the software and owns a license to use it. You receive no recurring income. Best for bespoke tools built to a specific client's specification.

Example: GHS 8,000 for a custom inventory system for a Kotei hardware shop, built, delivered, done

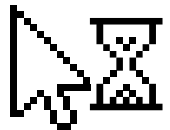




The Four Pricing Models

- **SaaS subscription:** client pays monthly or annually to access the software you host. Predictable recurring income. Best for multi-tenant platforms.

Example: GHS 250 per month per business for a cloud-based sales tracking platform.

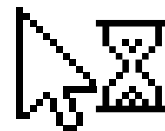




The Four Pricing Models

- **Freemium:** free tier attracts users; paid tier unlocks power features. Best for consumer-facing or B2C products.

Example: free up to 50 product listings; GHS 120 per month for unlimited listings and analytics.

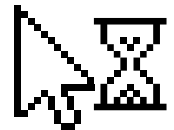




The Four Pricing Models

- **Usage-based:** charge per transaction, per API call, or per active user. Aligns cost directly to client value.

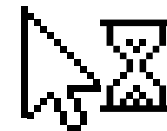
Example: GHS 0.80 per payment processed through your platform.





What Most Developers Forget to Price

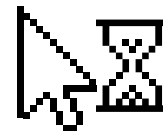
- **Discovery and requirements time:** the hours you spent interviewing the client and writing the requirements document are work. Price them.
- **Revision rounds:** how many rounds of changes are included? Every additional round beyond that is a paid change request.
- **Testing and QA time:** writing and running tests is not free. It typically adds 20–30% to development time.





What Most Developers Forget to Price

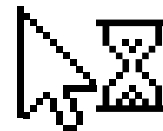
- **Deployment and configuration:** setting up hosting, environment variables, domain, and SSL takes time. Price it.
- **Documentation and handover:** the user guide, the README, the handover meeting, all of this is professional work.





What Most Developers Forget to Price

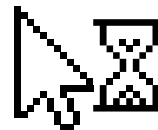
- **Support period:** how long will you fix bugs at no extra charge after delivery? 30 days? 90 days? State it explicitly.
- **Uncertainty margin:** add 15–20% to every estimate. Projects always take longer than expected. This is not greed; it is professional realism.





The Five-Step Pricing Method

- **Step 1, Cost Floor:**
 - Add up: your time (hours × hourly rate) + hosting costs + third-party service fees + revision rounds + support period.
 - This is the minimum you can charge without losing money





The Five-Step Pricing Method

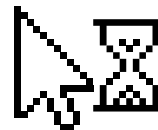
- **Step 2, Market Rate Research:**
 - What are similar tools selling for in Ghana?
 - What do Upwork developers charge for comparable work?
 - Anchor your price to real data, not guesswork





The Five-Step Pricing Method

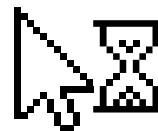
- **Step 3, Value-Based Pricing:**
 - What is this problem worth to the client?
 - If your system saves a business GHS 3,000 per month in wasted time, charging GHS 600 per month is a bargain, not an imposition.





The Five-Step Pricing Method

- **Step 4, Competitive Positioning:**
 - Are you premium, mid-market, or affordable?
 - Your price signals quality. Underpricing communicates low quality even when the work is excellent.

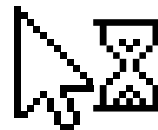




The Five-Step Pricing Method

- **Step 5, Client Validation:**

- Ask the client: 'What would you expect to pay for a solution to this problem?'
- Their answer is data. It does not bind you, but it informs you.





Pricing - Common Mistake vs Best Practice

X Common Mistake:

- Developer quotes GHS 1,500 for an inventory system because they feel uncomfortable charging more as a student.
- Three weeks into the project, the **client adds new requirements**. The developer keeps saying yes, not wanting to seem difficult.
- The project takes **eight weeks instead of three**. The developer earns less than GHS 200 per week for their time.
- The client is satisfied. The developer is **exhausted and resentful**.
- They have **accidentally** undervalued their profession.





Pricing – Common Mistake vs Best Practice

✓ **Best Practice:**

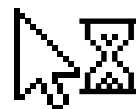
- Developer uses the five-step method. Cost floor calculates to GHS 2,800.
- Market rate for similar work is GHS 3,000 to 4,000. They quote GHS 3,200.
- The quote is broken into: GHS 1,600 **deposit** to start, GHS 960 at MVP delivery, GHS 640 on final acceptance.
- The contract specifies three revision rounds included. Any **additional change** is priced at GHS 150 per day.
- The developer earns **fairly**, the client knows exactly what to expect, and the relationship starts on a **professional footing**.





Worth Thinking (About Pricing)

- If you were charging for the system your team is building this semester, what would your cost floor be in Ghana cedis? Have you ever calculated it?
- If a client tells you your price is too high, what is the first thing you should do, and what is the one thing you should never do?
- What does it cost you in time and money to run your deployed application for one month? Does that number inform your pricing?
- At what point does a project become **unprofitable**? Are you tracking your hours against your initial estimate?
- Would you charge the same price for a sole trader in Kotei as you would for a multi-branch business in Accra? Why or why not?





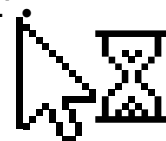
Contracts & **Getting Paid**





What Every Developer's Contract Must Contain

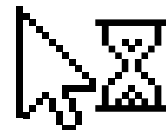
- **Scope of work:** exactly what you will build, described **precisely enough** that 'we agreed on X' cannot be disputed later. Reference the requirements document.
- **Payment terms:** amount, schedule (deposit → milestone → final), accepted payment methods, and **consequences of late payment**.
- **IP assignment:** who **owns** the code on delivery? Specify clearly. Default under Ghanaian law is the developer.





What Every Developer's Contract Must Contain

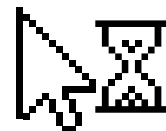
- **Change request process:** how are additions to scope handled? Who approves them? How are they priced?
- **Support period:** how long after delivery will you fix bugs at no charge? What is excluded from free support?
- **Termination clause:** what happens if either party wants to end the engagement early? What has already been paid for stays paid.





What Every Developer's Contract Must Contain

- **Limitation of liability:** you are not responsible for losses caused by the client's misuse of the software, or by factors outside your control (hosting outages, power cuts).
- **Electronic Transactions Act, 2008:** a contract signed digitally is legally binding in Ghana. Email confirmation of agreed terms can constitute a contract.

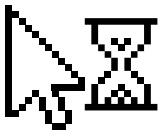




Fixed-Price vs Time-and-Materials: Know the Difference

Fixed-Price Contract:

- Client pays an agreed total regardless of how long the project takes
- All risk of underestimating sits with the developer
- Gives the client certainty and budget predictability
- Works well when the scope is clearly defined and unlikely to change
- **Your danger:** scope creep with no mechanism to charge for it
- **Best for:** well-defined projects with stable requirements

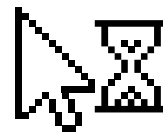




Fixed-Price vs Time-and-Materials: Know the Difference

Time-and-Materials (T&M) Contract:

- Client pays for actual time spent (at an agreed hourly or daily rate) plus materials
- Risk of cost overrun sits with the client
- Requires transparent, **trusted** time-tracking
- Works well when requirements are likely to **evolve**
- **Your danger:** a client who disputes your hours or drags out decisions
- **Best for:** ongoing work, exploratory projects, long-term engagements

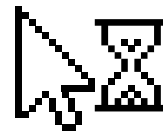




Deposits, Milestones, and What to Do When a Client Will Not Pay

The professional standard in Ghana:

- **30–50%** deposit before any significant work begins. This is not a sign of distrust, it is standard professional practice.
- A milestone payment at an agreed delivery point (typically the MVP). Agreed in the contract, not negotiated on the day.
- Final payment on formal client acceptance. The MVP Acceptance Form is your trigger for the final invoice.
- **Never begin significant work** without a deposit. This protects you from the most common exploitation of new developers.

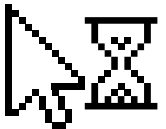




Deposits, Milestones, and What to Do When a Client Will Not Pay

When a client refuses to pay:

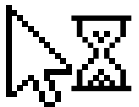
- Step 1: written payment reminder with the original invoice and contract attached.
- Step 2: formal demand letter stating the amount owed, the contract reference, and a deadline for payment.
- Step 3: **Alternative Dispute Resolution** (ADR) under Ghana's ADR Act, 2010, faster and cheaper than court.
- Step 4: Small Claims Court for amounts under GHS 10,000, simple process, no lawyer required.





Worth Thinking [About Contracts and Payments]

- If your current semester client wanted to continue working with you commercially after graduation, what contract would you need to put in place before starting?
- What is in your current MoU that protects you? What is missing that a real commercial contract would require?
- If a client paid you a deposit and then stopped responding to messages, what would you do, and at what point would you stop working?
- Have you and your team agreed on how you would divide payment if a client paid your team directly for this project?
- What is the earliest point in a client relationship at which you should introduce the topic of payment terms?





Change Management

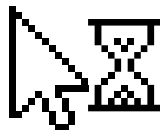




Defect vs Change: The Most Important Distinction in Client Work

Defect, your responsibility to fix

- The software does not do what was specified in the agreed requirements document
- The login page errors out on valid credentials, that was required to work
- The sales report shows wrong totals, that is a bug, not a new feature
- Fixing defects is included in your support period **at no extra charge**
- **The test:** does the requirements document say it should work this way? If yes, it is a defect.





Defect vs Change: The Most Important Distinction in Client Work

Change, a new piece of work

- The client wants something that was not in the original agreed scope
- 'Can you add a payroll module?', that was never in the requirements
- 'Can we change the color scheme to match our new branding?', not discussed,
- now requested
- Changes are priced separately and require written approval before work begins
- **The test:** is this in the requirements document? If no, it is a change.

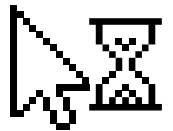




The Change Request Process – Never Skip a Step

- **Step 1, Assess:**

When a client requests a change, do not say yes or no immediately. Say: 'Let me assess what that involves and come back to you.'

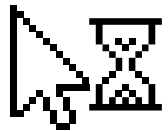




The Change Request Process – Never Skip a Step

- **Step 2, Document:**

Write a Change Request describing exactly what is being added or changed. Reference the original requirements to show why this is additional scope.

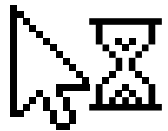




The Change Request Process – Never Skip a Step

- **Step 3, Price:**

Estimate the time required. Seemingly small changes often touch multiple parts of the system, assess the full impact before quoting. Add your uncertainty margin.

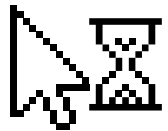




The Change Request Process – Never Skip a Step

- **Step 4, Get Written Approval:**

Send the Change Request document. The client must respond in writing (email is sufficient) confirming they accept the scope and cost. Never accept a verbal yes.

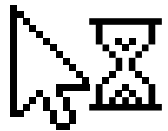




The Change Request Process – Never Skip a Step

- **Step 5, Build:**

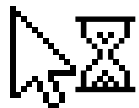
Only after written approval has been received do you begin work on the change. This sequence is non-negotiable.





Scope Freeze: How to Protect the Project and the Relationship

- Once requirements are agreed and signed off, that version of the scope is frozen for the current phase.
- New requests go into a backlog, not into the current sprint. This is not a rejection; it is professional project management.
- MoSCoW in practice: 'Won't Have this phase' is a promise to consider it in the next phase, not a permanent no.
- Tell the client: 'That is a great idea. We have logged it. Once we deliver what we have agreed, that becomes the first item we discuss for phase two.'
- The client who adds requirements freely and the developer who accepts them freely are both creating a disaster. A professional manages the boundary.





Worth Thinking About (Change Management)

- Think about your current client project. If your client came to class tomorrow and asked for a feature that was not in your requirements document, how would you respond, exactly, word for word?
- Have you and your team defined what a 'defect' means for your project? What is your support period after delivery?
- How much does it cost you to implement a change that touches the database schema, the API, and the frontend simultaneously? Have you ever calculated that?
- At what point does a client's change request become large enough to justify restarting the contract rather than amending it?
- If a client verbally approved a change and later refused to pay for it, what evidence would you have to support your position?





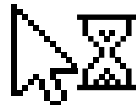
Winning & Keeping Clients





The First Impression Is a Technical Deliverable

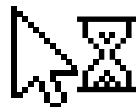
- Clients form an opinion of your team before they see a single line of code
- Punctuality to meetings, professionalism in emails, quality of written documents, all of these are signals the client is reading
- A Requirements Document delivered on time and formatted clearly says: this team is reliable
- A Requirements Document delivered late, formatted carelessly says: expect the same from the software
- The first meeting sets the tone for the entire engagement, arrive prepared, arrive on time, arrive having read your own notes from the previous meeting
- **Soft skills are not separate from engineering. They are part of the product the client is buying.**





Proactive Communication: The Simplest Client Retention Tool

- Clients become anxious when they do not hear from the development team. Anxiety becomes doubt. Doubt becomes dissatisfaction.
- A weekly two-sentence update costs almost nothing: 'This week we completed the login flow and started the product listing page. Next week we will integrate Paystack. No blockers currently.'
- Send it whether or not there is anything dramatic to report. The act of sending is the message.
- When there is a problem, communicate it before the client discovers it. A developer who says 'we found an issue and here is how we are fixing it' keeps trust. A client who discovers the issue first loses trust.
- WhatsApp is appropriate for brief updates in the Ghanaian market. Email for anything that requires a record. Never use only verbal communication for anything that matters.
- **The teams that clients rave about are rarely the most technically brilliant. They are the most reliable communicators.**

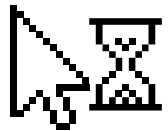




Handling Complaints: The Four-Step Response

- **Step 1, Acknowledge:**

‘I hear you, and I understand this is frustrating.’ Do not be defensive. Do not explain before you have acknowledged. The client needs to feel heard first.

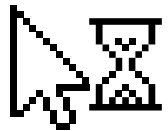




Handling Complaints: The Four-Step Response

- **Step 2, Own what is yours:**

If the error is the team's fault, say so clearly.
'This is our mistake and we are going to fix it.'
Accountability without excessive apology.

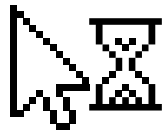




Handling Complaints: The Four-Step Response

- **Step 3, Explain what was outside your control:**

Hosting outages, third-party API failures, and client-side network issues are not your fault. Explain this calmly, with evidence. Do not blame; inform.



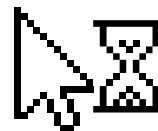


Handling Complaints: The Four-Step Response

- **Step 4, Propose a concrete next step:**

‘Here is what we will do and when it will be done.’ Give a specific time. Then deliver on it. A complaint well-handled often creates a more loyal client than no complaint at all.

The one thing never to do: disappear. Going silent during a problem is the fastest way to end a professional relationship.





Turning Delivery into a Long-Term Relationship

- The MVP handover is not the end of the client relationship, it is the moment the commercial relationship can begin
- At the handover meeting, present the 'What's Next' roadmap: all the Could Haves and Won't Haves from the MoSCoW list, now available as phase two
- Offer a hosting and support retainer: a monthly fee to keep the system running, updated, and secure. This converts a one-time project into recurring income.
- Referrals in the Ghanaian market: a satisfied Kejetia trader tells other traders. A clinic owner who has a working appointment system mentions it to a pharmacist. Your reputation travels faster than any advertisement.
- Ask for the referral explicitly: 'If you know anyone else who might benefit from something like this, I would appreciate the introduction.' Most satisfied clients will say yes if asked directly.
- **The best marketing a Ghanaian software developer can invest in is quality of delivery and professionalism of conduct.**





About Winning and Keeping Clients

- When did you last send your client an update? What did it say? If you have not sent one recently, what would it say if you sent it right now?
- If your client called you angry because the system was down, what would you say in the first thirty seconds?
- What is one thing your team could do this week, beyond writing code, that would make your client more confident in your ability to deliver?
- At the handover meeting at the end of Week 6, what will you present as 'phase two' of this project, and how would you price it?
- Who else in the KNUST environs might benefit from a system like the one you are building? Could your current client make that introduction?





The Career Arc

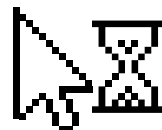


Stage 1: The Student Developer

Years 3-4 of study, and the first year after graduation

What to focus on:

- Build a public portfolio on GitHub: real projects, clean READMEs, evidence of your thinking
- Contribute to open-source projects, even small pull requests demonstrate professionalism
- Take on small paid projects: church websites, student union systems, NGO tools, client-facing experience is irreplaceable
- Document every project: what problem it solved, what you built, what you learned, what you would do differently
- Attend community events: GDG Kumasi, Accra Tech Week, KNUST alumni meetups, your first job will likely come through a connection, not an application
- Learn to price: even if you are charging very little at first, practice the discipline of calculating cost floor and market rate



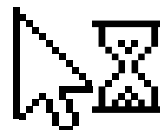


Stage 1: The Student Developer

Years 3-4 of study, and the first year after graduation

Pitfalls to avoid:

- Building projects no one ever uses: prioritize things that solve real problems for real people
- Working for free indefinitely: unpaid work is valuable experience; unbounded unpaid work is exploitation, know the difference
- Hiding your work: imperfect code on GitHub is better than no code anywhere
- Waiting until you feel 'ready': you will never feel fully ready. Start with what you have.
- Neglecting soft skills: the ability to communicate clearly with a client is as rare and valuable as technical skill



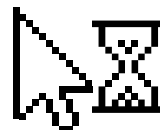


Stage 2: The Professional Developer

Years 1-5 in industry: employed, freelance or both

What to focus on:

- Deepen one technical specialism: backend systems, mobile, data engineering, DevOps, generalists are valuable; specialists are in demand
- Understand the business you are working in, not just the code: what does the company sell? How does your work affect that?
- Learn to estimate accurately: track your estimates vs actuals on every project; the gap will shrink over time
- Build a professional network deliberately: stay in contact with former colleagues, contribute to industry communities
- Start thinking about products: identify a problem you see repeatedly in your work that software could solve at scale
- If freelancing: formalize everything, registered business, proper contracts, separate bank account, tax compliance



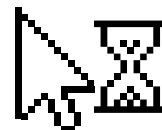


Stage 2: The Professional Developer

Years 1-5 in industry: employed, freelance or both

Pitfalls to avoid:

- Staying in your comfort zone technically: the skills that got you hired are not the skills that will get you promoted
- Treating every job as temporary before you have learned everything it has to teach you
- Neglecting financial literacy: understand your taxes, your pension, your rates
- Underpricing yourself because you are not yet confident: research market rates and charge them
- Burning bridges: the software industry in Ghana is small; how you leave a job matters as much as how you performed in it



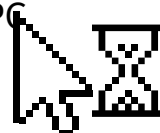


Stage 3: The Founder

Starting and running your own software business

What to focus on:

- Solve a problem you understand from experience: the best products are built by people who have lived the problem
- Start with services (consulting, bespoke development) to generate cash flow before investing in a product
- Document everything from day one: contracts, ADRs, client communications, financial records
- Hire slowly and for culture as much as skill: one toxic team member destroys more value than their technical output creates
- Build recurring revenue as early as possible: subscriptions and retainers are more stable than one-time projects
- Register your business, your IP, and your trademarks early: the GIPC and Registrar-General's Department exist for this



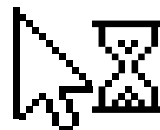


Stage 3: The Founder

Starting and running your own software business

Pitfalls to avoid:

- Starting a company before you understand how to find and keep clients: technical excellence without sales ability is a very expensive hobby
- Trying to do everything yourself: legal, accounting, design, sales, and engineering is too much for one person
- Neglecting the legal foundations: informal arrangements work until they don't, then they are catastrophically expensive to resolve
- Scaling too early: hire, build, and invest only when demand requires it, not in anticipation of demand that has not yet arrived
- Confusing revenue with profit: many Ghanaian tech businesses fail because the founder does not know their actual margin





About Your Career Arc

- If you graduated tomorrow, what would your portfolio look like? Would a technical hiring manager or a first client find enough evidence to trust you?
- What is the one technical skill you most want to deepen before graduation, and what is your plan to do that?
- Do you know what a software engineer at your level earns in Accra? In Lagos? On Upwork? If not, why not?
- If you were to take on a paid freelance project in the next six months, what would you need to have in place before you started?
- Five years from now, employed, freelancing, or founder? What would make one of those paths more likely than the others?





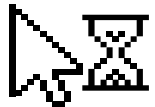
Scaling the Decision





When to Scale a Product

- **The signal is demand, not ambition.**
- Scale when your system is straining under real user load, not in anticipation of users you do not yet have
- Indicators that scaling is needed: response times degrading under normal load; database queries taking seconds instead of milliseconds; users experiencing errors during busy periods
- The wrong reason to scale: because the architecture looks inelegant, or because a blog post said micro services are better



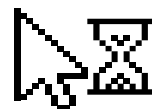


When to Scale a Product

- **Sequence of scaling decisions:**

1. Optimize first: fix slow queries, add caching, compress assets. These are free or cheap.
2. Scale vertically: upgrade to a larger server. Simple, no code changes required.
3. Scale horizontally: add more servers behind a load balancer. Requires stateless application design.
4. Decompose: extract the heaviest module into its own service. Only when the above options are exhausted.

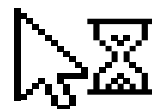
For your semester project: after the MVP, the scaling question is 'what would break first at 10,000 users?', not 'should we rewrite in micro services?'





When to Scale a Business

- **Scale when demand exceeds your capacity to deliver at your current quality standard.**
- If you are regularly turning away client work because you do not have enough time, that is a signal to hire
- If a single client accounts for more than 50% of your revenue, that is a risk, diversify before scaling
- Before hiring anyone: document your processes. A new team member needs to be able to follow what you have built without you explaining everything verbally.

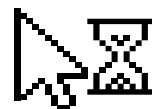




When to Scale a Business

Signs you are scaling too early:

- You are hiring before you have consistent, profitable revenue
- You are building a team before you have a repeatable way to find clients
- You are renting office space before the rent is a small fraction of your monthly income





When to Scale a Business

Signs you are scaling too late:

- You are regularly missing deadlines because you are overloaded
- You are losing clients to competitors who can take on more work
- Your quality is declining because you are stretching across too many projects simultaneously





About Scaling

- For your current project: if 500 people tried to use the system simultaneously this evening, what would fail first?
- At what revenue level or user count would you say your semester project is 'ready to scale'? What is that based on?
- If you were running a freelance software business and a client offered you a contract three times larger than anything you had done before, what would you need to assess before accepting?
- What is the difference between a developer who is busy and a developer who is growing? Which one are you building habits for?
- If a Ghanaian investor offered you GHS 50,000 to scale the product you are building this semester, what would you spend it on, and in what order?





What Separates a Good Developer From a Great One

The things that separate a good developer from a great one are rarely technical:

- They know who owns what they build.
- They price their work to reflect its value.
- They put agreements in writing.
- They communicate before the client has to ask.
- They handle complaints without losing composure.
- They turn one project into the next conversation.

